

Week 4 Exercises Tuples, Lists and Loops

1. Save the word 'computing' in a variable. Using slicing, print only 'put'.
2. Given a list of integers and a number n , find the n th largest element in the list. The element will be stored in the `nth_max` variable. You are allowed to use all list operations and built in functions; your code should work for any value of N (assuming N isn't larger than the list length). Try to make this into a function: `find_nth_max(list, n)` and the list is: `mylist = [30, 21, 16, 66, 78, 109, 1, 4, 52]`
3. Write a program that will unpack a tuple into separate variables. Example: `tuple = (1,3,5,7)` will result in creating 4 variables, each containing one of the tuple elements above.
4. Modify your code for exercise above such that it creates a new tuple containing the squared values of the original elements, in the same order. E.g. `new_tup = (1,9,25,49)`
5. Given a list of tuples produce a new list of tuples where each tuple element is multiplied by the tuple's index in the list. `L = [(2,4,6), (1,3,5),(10,20,30)]`. Your output should look like `[(0,0,0), (1,3,5),(20,40,60)]`.
6. Given two lists: `[1,2,3,4,5]` and `[1,2,3,4,5]`, use for loops to print the result from multiplying each value of list 1 with each value of list 2, e.g.: `1*1, 1*2, 1*3... 2*1, 2*2, ... 5*5`.
7. (Advanced) Given a string containing only square brackets, `[]`, you must check whether the brackets are balanced or not. The brackets are said to be balanced if, for every opening bracket, there is a closing bracket. You will write your code in the `check_balance()` function, which returns `True` if the brackets are balanced and `False` if they are not. For an empty string, the function will return `True`. For the sake of simplicity, you can assume that the string will not contain any other characters. Examples:
 - a. `[ [ [ ] ] ]` – balanced -> `True`;
 - b. `[[][]]` – balanced -> `True`;
 - c. `[[]` – unbalanced -> `False`;
 - d. `][][]` - unbalanced -> `False`.